

Definition of Done

Product Name: Vision Studio Spatial Layout Engine

Team Name: Vision Studio

Date: Apr 10, 2026

Last Modified: Jun 6, 2026

General Definition of Done

A task or user story is considered done when all of the following are true:

1. The requested feature, fix, or document has been completed according to the sprint plan or team agreement.
2. The work supports the related user story and product goal.
3. The work is integrated into the appropriate part of the project.
4. The work has been manually tested or reviewed.
5. The work does not introduce obvious runtime errors, broken pages, or blocking bugs.
6. Any major known limitations are documented.
7. The work is committed to the repository or added to the release documentation.
8. The team agrees that the work is ready to count as completed.

User Story Completion Criteria

A user story is considered done when:

1. The user-facing goal of the story is implemented.
2. The user can complete the intended workflow from the UI or system interface.
3. The story's acceptance criteria have been met.
4. The feature has been tested in the context of the full application, not only in isolation.
5. The feature works with related parts of the system, such as the editor, backend, database, AI service, furniture catalog, 3D preview, or export flow, where applicable.
6. Edge cases and limitations have been considered.
7. Known problems are added to the Test Plan and Report or Release Summary if they cannot be fixed before release.

Code Completion Criteria

Code is considered done when:

1. The feature or fix is implemented in the correct application area, such as client/, server/, python/, or supabase/.
2. The code follows the project's existing structure and naming conventions.
3. The code is readable and organized.

4. Unused debug code, temporary console logs, and unnecessary commented-out code are removed when possible.
5. The code does not break existing core workflows.
6. The code can run locally with the documented setup instructions.
7. Required environment variables are documented in the README or environment template files.
8. Any new dependency is added to the correct dependency file, such as package.json or requirements.txt.

Frontend Definition of Done

Frontend work is considered done when:

1. The feature appears correctly in the user interface.
2. The page or component is responsive enough for normal use.
3. The UI matches the general Vision Studio style and layout.
4. The user can understand what action to take next.
5. Buttons, forms, inputs, and navigation elements work as expected.
6. The feature handles empty states, loading states, and error states where practical.
7. The feature does not create obvious console errors during normal use.
8. The feature has been manually tested in the browser.
9. Automated tests are added or updated when the feature is suitable for testing.

Backend Definition of Done

Backend work is considered done when:

1. The API route, service, or database operation supports the required frontend or system behavior.
2. The backend returns clear responses for successful and failed requests.
3. The backend handles missing or invalid input where practical.
4. The backend does not expose sensitive keys or credentials.
5. The backend uses environment variables for configuration instead of hard-coded secrets.
6. The backend has been tested through automated tests, API smoke tests, and manual integration testing.
7. The frontend can successfully call the backend endpoint when the services are configured correctly.

AI/Python Service Definition of Done

AI or Python service work is considered done when:

1. The Python/FastAPI service can start successfully with the documented dependencies.
2. The service supports the intended AI or parsing workflow.
3. The frontend or backend can call the service through the expected API flow.

4. The service handles failed parsing, missing inputs, or unavailable external services without crashing the full application.
5. Known limitations of AI parsing or recognition accuracy are documented.
6. The workflow has been manually tested with at least one realistic input.

Database and Save/Load Definition of Done

Database-related work is considered done when:

1. The required table, relationship, or schema change is present in the Supabase schema or setup documentation.
2. Save/load behavior works when Supabase credentials and schema are configured correctly.
3. The app preserves important project state, such as room layout data, placed furniture, and relevant project information.
4. The app handles database errors or missing configuration as gracefully as possible.
5. Any Supabase setup requirements are documented for the final release.

Furniture and Layout Definition of Done

Furniture and layout features are considered done when:

1. Furniture items can be displayed in the catalog.
2. Furniture items include useful information such as name, category, dimensions, and preview, where available.
3. Users can place furniture into the room layout.
4. Users can move, rotate, or resize furniture where supported.
5. Furniture state is stored in the layout state.
6. Fit and collision checks identify major placement problems.
7. The system provides a warning or feedback when furniture overlaps, is too large, or does not fit the room.
8. Known limitations with complex layouts or custom dimensions are documented.

3D Visualization Definition of Done

3D visualization work is considered done when:

1. The user can open a 3D preview from the room design workflow.
2. The 3D preview renders the room structure at an MVP level.
3. Furniture appears in the 3D scene as models or placeholder geometry where supported.
4. The user can navigate or inspect the room from different angles.
5. The 3D view does not block the rest of the app from functioning.
6. Known limitations around model quality, realism, lighting, alignment, or external export compatibility are documented.

Testing Definition of Done

Testing is considered done when:

1. The feature has been manually tested through the user workflow.
2. Related automated tests are run when available.
3. New automated tests are added when practical and time allows.
4. The client test suite passes before final release.
5. The server smoke tests pass before final release.
6. Build verification is completed before final release.
7. Any failing, skipped, or missing tests are documented in the Test Plan and Report.
8. Known bugs are documented rather than hidden.

For the final release, the team verified:

1. Client Vitest suite passed.
2. Server e2e smoke tests passed.
3. Client production build succeeded.
4. Python service behavior was tested through manual and integration workflows, but formal Python unit tests were not included.

Documentation Definition of Done

Documentation is considered done when:

1. The document has a clear title, product name, team name, date, and last modified date when appropriate.
2. The document is stored in the correct release folder.
3. The document is written clearly enough for a professor, TA, or future developer to understand.
4. The document reflects the actual final state of the project.
5. The document does not claim unsupported or unfinished functionality as complete.
6. Known bugs, limitations, and shortcuts are described honestly.
7. Setup instructions, user steps, or design explanations are specific enough to be useful.
8. The document is linked or referenced from the README or final submission documentation index when possible.

Pull Request and Review Criteria

A code change is ready to merge when:

1. The change is focused on a clear task, bug fix, or feature.
2. The developer has tested the change locally.
3. The change does not obviously break the homepage, upload flow, editor, save/load, furniture placement, 3D preview, or export flow.
4. Merge conflicts have been resolved.

5. The pull request or commit description explains what changed.
6. A teammate has reviewed or checked the work when possible.
7. Any remaining known issues are documented before release.

Release Branch Definition of Done

The final release branch is considered done when it includes:

1. Source code needed to install and run the system.
2. Test code that was created for the project.
3. Scrum documents, including release plan, sprint plans, sprint reports, team working agreement, Definition of Done, and style guide.
4. Test Plan and Report.
5. Release Summary with key user stories, acceptance criteria, known problems, and product backlog.
6. Release documents, including installation instructions, user guide, design/architecture documentation, dependencies, and deployment notes, where needed.
7. Environment templates without real secrets.
8. Final test results documented.
9. Known bugs and limitations documented.
10. Temporary or non-submission files are removed when possible.